

CLI Agent Release and Implementation Readiness Gate v0.1

Release-complete and implementation-ready gate profile for the C-Governed CLI Agent Mesh package

Kotov Ivan
Bruxelles 2026

Status	Draft package-control profile v0.1
Date	2026-05-17
Package	C-Governed CLI Agent Mesh
Layer	c = a + b / SER / L4 / Agent Governance / CLI Worker Mesh / Release Hygiene / Implementation Readiness / Witness / Conformance
Document class	readiness gate profile / release-control artifact / implementation-handoff artifact / package-control companion
Assertion class	C-A10 package-control artifact; C-A7 where witness, hash, canonicalization, schema verification, or conformance-evidence claims are made
Primary boundary	the package must not claim release-complete, implementation-ready, public-safe, deployment-ready, provider-safe, or conformance-passed status until the required gates are satisfied, recorded, and challengeable.

Contents

0. Executive definition	5
1. Purpose	5
2. Non-goals.....	6
3. Corpus bridge set	7
3.1 Explicit bridge: $c = a + b$	7
3.2 Quiet bridge I: L4 and irreversible publication	7
3.3 Quiet bridge II: information theory and claim compression.....	7
3.4 Quiet bridge III: engineering stage gates.....	7
3.5 Earth paragraph	7
4. Core doctrine	8
4.1 Primary doctrine	8
4.2 Readiness axioms.....	8
5. Definitions.....	9
5.1 Draft-complete architecture	9
5.2 Release-complete	9
5.3 Implementation-ready.....	9
5.4 Implementation-executed	9
5.5 Conformance-supported	9
5.6 Deployment-ready.....	10
6. Readiness state taxonomy	10
6.1 State ordering.....	10
6.2 Non-equivalence rule.....	11
7. Claim control vocabulary.....	11
7.1 Allowed claim classes	11
7.2 Prohibited claims before gate.....	11
8. Gate A — Release-complete.....	12
8.1 Gate definition	12
8.2 Required release-complete conditions.....	12
8.3 Release-complete pass criteria	13
8.4 Release-complete failure conditions	14
8.5 Allowed claims after Gate A.....	14
9. Gate B — Implementation-ready	14

9.1 Gate definition	14
9.2 Required implementation-ready conditions	15
9.3 Implementation-ready pass criteria	15
9.4 Implementation-ready failure conditions	16
9.5 Allowed claims after Gate B	16
10. Gate C — Implementation-executed	16
10.1 Gate definition	16
10.2 Required implementation-executed artifacts	16
10.3 Gate C caution	17
11. Gate D — Conformance-supported	17
11.1 Gate definition	17
11.2 Required conformance-supported artifacts	17
11.3 Conformance-supported limitation	18
12. Required companion documents before Gate B	18
13. Release/public surface requirements	19
13.1 Discoverability rule	19
13.2 Artifact ordering rule	19
13.3 No premature hash rule	20
14. Public / restricted handling	20
14.1 Default publication rule	20
14.2 Public-capable material	20
14.3 Restricted material	20
14.4 Restricted document warning	21
15. Machine-readiness requirements	21
15.1 JSON Schema is necessary but insufficient	21
15.2 Minimum machine artifacts before implementation execution	21
15.3 Semantic rules that MUST NOT be left to JSON Schema alone	22
16. Readiness record object	22
16.1 YAML shape	22
16.2 Decision values	23
17. Release completion record object	23
17.1 YAML shape	23
18. Implementation handoff record object	24
18.1 YAML shape	24
19. Claim control record object	25

19.1	YAML shape	25
20.	Current package assessment template.....	26
20.1	Current known state before this profile is integrated	26
20.2	Required update rule	26
21.	Stop conditions	26
22.	Minimal Codex handoff rule	27
23.	Minimal next-step sequence	28
24.	Readiness checklist summary.....	28
24.1	Release-complete checklist.....	28
24.2	Implementation-ready checklist	29
24.3	Conformance-supported checklist	29
25.	Closing rule.....	29

0. Executive definition

CLI Agent Release and Implementation Readiness Gate defines the conditions under which the C-Governed CLI Agent Mesh package may move from:

```
draft protocol pack
-> release-complete document package
-> implementation-ready handoff package
-> implementation-executed package
-> conformance-supported package
```

This profile exists to prevent a common failure mode:

```
conceptual coherence
  mistaken for
release readiness
  mistaken for
implementation readiness
  mistaken for
deployment safety
```

These are different states.

Compact formula:

```
A coherent doctrine is not a release.
A release is not an implementation.
An implementation is not conformance.
Conformance is not deployment safety.
```

The gate profile controls what may be honestly claimed, what remains blocked, what must be patched, what must be machine-readable, and what must remain restricted.

1. Purpose

The C-Governed CLI Agent Mesh package has moved beyond a single essay. It now contains protocol documents, profiles, registries, red-line boundaries, memory-gate rules, witness rules, rollback semantics, conformance fixtures, and release notes.

The package therefore needs a readiness gate that answers:

1. When is the package **release-complete**?

2. When is the package **implementation-ready**?
3. What claims are allowed before each gate?
4. What claims are prohibited before each gate?
5. Which missing artifacts block release?
6. Which missing artifacts block implementation handoff?
7. Which sensitive documents require restricted handling?
8. Which machine-readable objects must exist before coding begins?
9. Which evidence is needed before conformance can be claimed?
10. Which failures force hold, freeze, rollback, or claim downgrade?

This profile is not decorative. It is a control surface for preventing premature publication, premature automation, and premature confidence.

2. Non-goals

This profile does not:

1. certify legal compliance;
2. certify product safety;
3. certify provider-specific cloud handling;
4. replace security review;
5. replace counsel review;
6. replace conformance testing;
7. replace implementation validation;
8. authorize deployment;
9. authorize public release of restricted material;
10. authorize offensive cyber activity;
11. authorize hack-back, live counter-operation, malware behavior, credential theft, evasion, unauthorized scanning, or autonomous retaliation;
12. make Markdown documents equivalent to executable validators;
13. make schema validity equivalent to system safety.

A readiness gate controls claims.

It does not magically make the system safe.

3. Corpus bridge set

3.1 Explicit bridge: $c = a + b$

In $c = a + b$, the readiness gate belongs to b : it is a procedural, documentary, machine-facing control artifact.

It protects c by preventing worker agents, release tooling, public packaging, or implementation scripts from treating incomplete documentation as authority.

The readiness gate does not become c .

It tells c , the human anchor, reviewers, and implementation agents when a transition is permitted, blocked, partial, or unsupported.

3.2 Quiet bridge I: L4 and irreversible publication

A public release is an L4 transition. It consumes time, creates public interpretation, propagates copies, affects reputation, and cannot be fully retracted. Therefore, release-complete status requires more than conceptual coherence. It requires discoverability, redaction, integrity, status hygiene, and claim discipline.

3.3 Quiet bridge II: information theory and claim compression

A label such as `implementation-ready` compresses many assumptions. If the assumptions are not explicit, the label leaks ambiguity into the system. This profile expands those compressed labels into checkable conditions.

3.4 Quiet bridge III: engineering stage gates

Engineering projects separate design review, fabrication release, installation, commissioning, and operational acceptance. A drawing can be approved while a machine is not yet built. A machine can be built while not yet certified for production. This profile applies the same staged discipline to CLI-agent governance.

3.5 Earth paragraph

On a real site, a folder of drawings is not the same thing as a finished electrical cabinet. A cabinet wired on the bench is not the same thing as a commissioned installation. And a commissioned installation is not the same thing as a building handed over to occupants. Each stage has a checklist, a responsible gate, and a signature. This document is that checklist for the CLI-agent package. Without it, somebody will eventually say “ready” when they only mean “looks coherent”. That is how small documentation mistakes become expensive smoke.

4. Core doctrine

4.1 Primary doctrine

Do not claim a stronger state than the package can prove.

4.2 Readiness axioms

ID	Axiom	Requirement
RG-AX-01	Coherence is not release readiness	A contradiction pass does not by itself make the package release-complete.
RG-AX-02	Release is not implementation	A public or archival release does not by itself make the package machine-executable.
RG-AX-03	Implementation-ready is not implemented	Implementation-ready means ready for disciplined implementation handoff, not that code already exists.
RG-AX-04	Implementation is not conformance	A validator or runner must produce evidence before conformance is claimed.
RG-AX-05	Conformance is not deployment safety	Passing internal conformance tests does not certify all real environments.
RG-AX-06	Claims must be bounded	Every readiness claim MUST specify scope, version, artifacts, evidence, and exclusions.
RG-AX-07	Sensitive material defaults restricted	Unknown publication sensitivity defaults to restricted handling.
RG-AX-08	Machine objects require schemas	Implementation handoff SHOULD NOT proceed without canonical object registry and schema extraction targets.
RG-AX-09	Fixtures are not test runs	Fixture existence does not equal conformance execution.
RG-AX-10	Missing witness fails closed	Privileged readiness transitions require witness or explicit hold.
RG-AX-11	Stale blockers must be patched	A register claiming missing files that now exist must be updated before release-complete status.
RG-AX-12	Red lines override all gates	No readiness gate can authorize prohibited behavior.

5. Definitions

5.1 Draft-complete architecture

A state where the main concepts, boundaries, profiles, and invariants are documented and no known hard contradiction blocks further packaging.

Draft-complete architecture does not imply release-complete or implementation-ready status.

5.2 Release-complete

A document-package state where the public/restricted split, status registers, release notes, reading order, red-line wording, file inventory, publication warnings, and release mechanics are synchronized enough for a controlled public, archival, or restricted release.

Release-complete does not mean implementation-ready.

5.3 Implementation-ready

A handoff state where an implementation worker such as Codex can receive precise, bounded, machine-facing tasks without inventing missing architecture.

Implementation-ready requires object registry, schema extraction targets, validator expectations, fixture manifest model, evidence packet model, local checker expectations, and claim controls.

Implementation-ready does not mean the code exists.

5.4 Implementation-executed

A state where Codex or another executor has generated schemas, manifests, validators, fixtures, checkers, or other machine artifacts according to the implementation-ready package.

Implementation-executed does not mean conformance-passed.

5.5 Conformance-supported

A state where tests have run, results are recorded, evidence packets exist, red-line cases are handled, and claims are supported by test outputs.

Conformance-supported does not mean deployment-certified.

5.6 Deployment-ready

A separate, later operational state requiring environment-specific security review, legal review where applicable, provider review, secret handling review, backup/rollback review, and monitored rollout.

Deployment-ready is out of scope for this v0.1 readiness profile.

6. Readiness state taxonomy

Readiness state IDs use prefix `RG-*`.

State	Name	Meaning	Claim strength
RG-0-DRAFT	Draft	Material exists but gates are not satisfied	weak
RG-1-DRAFT-COMPLETE	Draft-complete architecture	Conceptual package coherent at draft level	bounded architecture claim
RG-2-RELEASE-CANDIDATE	Release candidate	Most release blockers addressed; final review pending	candidate only
RG-3-RELEASE-COMPLETE	Release-complete	Controlled release package ready within declared scope	document release claim
RG-4-IMPLEMENTATION-READY	Implementation-ready	Machine-facing handoff is precise enough for Codex/executor	implementation handoff claim
RG-5-IMPLEMENTATION-EXECUTED	Implementation-executed	Machine artifacts generated and checked structurally	implementation artifact claim
RG-6-CONFORMANCE-SUPPORTED	Conformance-supported	Tests run and evidence supports claims	test-backed claim
RG-X-BLOCKED	Blocked	A blocker prevents claim escalation	no escalation

6.1 State ordering

```
RG-0-DRAFT
-> RG-1-DRAFT-COMPLETE
-> RG-2-RELEASE-CANDIDATE
-> RG-3-RELEASE-COMPLETE
-> RG-4-IMPLEMENTATION-READY
```

-> RG-5-IMPLEMENTATION-EXECUTED
-> RG-6-CONFORMANCE-SUPPORTED

6.2 Non-equivalence rule

RG-1 ≠ RG-3
RG-3 ≠ RG-4
RG-4 ≠ RG-5
RG-5 ≠ RG-6
RG-6 ≠ deployment certification

7. Claim control vocabulary

7.1 Allowed claim classes

Claim classes use prefix RGC-*.

Claim class	Meaning	Allowed when
RGC-DRAFT	draft protocol material	RG-0 or above
RGC-DRAFT-COMPLETE	architecture is draft-complete	RG-1 or above
RGC-RELEASE-CANDIDATE	release candidate under review	RG-2 or above
RGC-RELEASE-COMPLETE	release-complete document package	RG-3 or above
RGC-IMPLEMENTATION-READY	ready for implementation handoff	RG-4 or above
RGC-IMPLEMENTED	implementation artifacts exist	RG-5 or above
RGC-CONFORMANCE-SUPPORTED	conformance claims supported by evidence	RG-6 or above
RGC-DEPLOYMENT-READY	environment-specific deployment approval	out of scope / future gate

7.2 Prohibited claims before gate

Before RG-3 , the package MUST NOT claim:

release-complete
public-safe
archival-ready
final package
complete release

Before RG-4 , the package MUST NOT claim:

```
implementation-ready
ready for Codex implementation
machine-ready
schema-ready
validator-ready
```

Before RG-5, the package MUST NOT claim:

```
implemented
validator exists
schemas generated
fixtures installed
checker implemented
```

Before RG-6, the package MUST NOT claim:

```
conformance passed
operational conformance
red-line tests passed
evidence-backed safety
```

At no stage in v0.1 may the package claim:

```
deployment certified
legally certified
provider-safe in all environments
production-safe
universal AI-agent safety standard
```

8. Gate A — Release-complete

8.1 Gate definition

RG-3-RELEASE-COMPLETE means the package is ready as a controlled document release within declared public, restricted, or archival scope.

It does not mean the package is machine-executable.

8.2 Required release-complete conditions

ID	Condition	Required state
GA-001	Canonical file inventory exists	complete

ID	Condition	Required state
GA-002	Package index matches actual files	complete
GA-003	README matches package state	complete
GA-004	Release notes match package state	complete
GA-005	Open issues register is updated	complete
GA-006	Contradiction register is updated	complete
GA-007	No stale “missing file” claims remain	complete
GA-008	Public/restricted/internal split is defined	complete
GA-009	Sensitive files are classified	complete
GA-010	Red-line language pass is complete	complete
GA-011	Raw evidence handling is centralized or explicitly deferred with blocker	complete or blocker recorded
GA-012	Release/public surface rules are defined or explicitly deferred with blocker	complete or blocker recorded
GA-013	Final reading order is explicit	complete
GA-014	Protected claims are controlled	complete
GA-015	Freeze point for Markdown source is declared	complete
GA-016	PDF generation is blocked until Markdown freeze	complete
GA-017	SHA256 generation is blocked until all final artifacts exist	complete
GA-018	Release tag is blocked until final review	complete
GA-019	Public package excludes restricted operational material	complete
GA-020	Release decision is witnessed or recorded	complete

8.3 Release-complete pass criteria

A package may pass Gate A only if:

```
all GA conditions are complete;
all unresolved blockers are explicitly non-blocking for the declared release scope;
restricted material is not silently included in public output;
claim vocabulary is updated across package-control files;
release notes do not overclaim implementation or conformance;
public readers can find the correct entry point and reading order.
```

8.4 Release-complete failure conditions

Gate A fails if any of the following remain true:

1. package index references missing canonical files without status explanation;
2. release notes claim blockers that have already been resolved;
3. open issues still list created files as absent;
4. public/restricted split is not applied;
5. DEB / IRP / SCDP are published publicly without redaction review;
6. red-line wording differs enough that a reader could infer hack-back, retaliation, credential theft, live external action, direct memory write, or self-approval is allowed;
7. release artifacts are generated before source freeze;
8. SHA256SUMS are generated before final artifact set is closed;
9. release tag is created before final review;
10. README, release notes, and package index disagree about status.

8.5 Allowed claims after Gate A

After Gate A, the package may say:

```
release-complete document package
controlled release package
public/restricted split applied
architecture draft packaged for release
not implementation-ready unless Gate B also passes
not conformance-passed unless Gate F also passes
```

9. Gate B — Implementation-ready

9.1 Gate definition

RG-4-IMPLEMENTATION-READY means the package is ready for disciplined implementation handoff to Codex or another bounded executor.

It means:

```
Codex should implement from defined objects, schemas, validators, fixtures, and gates.
Codex should not invent missing architecture.
```

It does not mean implementation has already been executed.

9.2 Required implementation-ready conditions

ID	Condition	Required state
GB-001	Canonical object registry exists	complete
GB-002	Schema extraction target list is complete	complete
GB-003	Schema folder layout is defined	complete
GB-004	Stable schema \$id convention is defined	complete
GB-005	SCHEMA_INDEX.json model is defined	complete
GB-006	Package manifest model is defined	complete
GB-007	Semantic validator rules are defined	complete
GB-008	Local checker profile is defined	complete
GB-009	Agent registry profile is defined	complete
GB-010	Fixture manifest model is defined	complete
GB-011	Expected-result model is defined	complete
GB-012	Evidence packet model is defined	complete
GB-013	Conformance runner requirements are defined	complete
GB-014	Red-line test behavior is defined	complete
GB-015	Fail-closed behavior is machine-checkable	complete
GB-016	Witness event minimums for implementation are defined	complete
GB-017	Memory gate implementation boundaries are defined	complete
GB-018	Provider-specific assumptions are either profiled or explicitly deferred	complete
GB-019	Raw evidence sidecar handling is defined	complete
GB-020	Codex handoff constraints are written	complete

9.3 Implementation-ready pass criteria

A package may pass Gate B only if:

```
Codex can be given bounded implementation tasks without deciding architecture;  
all canonical objects have extraction targets or explicit deferral;  
validator behavior is specified enough to test PASS/FAIL/BLOCKED/HELD/QUARANTINED  
states;  
fixtures have manifest and expected-result structure;  
evidence packets have required fields;  
red-line behavior cannot be interpreted as optional;  
implementation claims remain weaker than conformance claims.
```

9.4 Implementation-ready failure conditions

Gate B fails if any of the following remain true:

1. schemas are mentioned but object registry is missing;
2. schema extraction plan exists but no canonical object list is complete;
3. fixture pack exists but no fixture manifest / expected-result / evidence-packet structure is implementation-ready;
4. local checker expectations are vague;
5. registry of agents is absent;
6. raw evidence handling is left to implementer judgment;
7. semantic rules are not separated from JSON Schema rules;
8. Codex must infer red-line behavior from prose alone;
9. provider-specific assumptions are neither profiled nor clearly deferred;
10. implementation task scope would allow Codex to edit protected state without a gate.

9.5 Allowed claims after Gate B

After Gate B, the package may say:

```
implementation-ready handoff package
ready for bounded Codex implementation
schema extraction and validator implementation may proceed
fixtures and evidence-packet implementation may proceed
not implemented until Gate C passes
not conformance-supported until Gate D passes
```

10. Gate C — Implementation-executed

10.1 Gate definition

RG-5-IMPLEMENTATION-EXECUTED means the implementation worker has produced the machine artifacts required by Gate B.

This gate is usually reached after Codex execution.

10.2 Required implementation-executed artifacts

ID	Artifact	Required state
GC-001	schemas/ directory	exists
GC-002	JSON Schema files	generated

ID	Artifact	Required state
GC-003	schemas/SCHEMA_INDEX.json	generated
GC-004	package manifest	generated
GC-005	fixture manifest	generated
GC-006	expected-result files	generated
GC-007	local checker	generated or documented
GC-008	semantic validator rules	implemented or mapped
GC-009	evidence packet templates	generated
GC-010	implementation report	generated

10.3 Gate C caution

Implementation-executed is still not conformance-supported.

Generated artifacts may be malformed, incomplete, or semantically wrong until tested.

11. Gate D — Conformance-supported

11.1 Gate definition

RG-6-CONFORMANCE-SUPPORTED means tests have run and produced evidence supporting declared conformance claims.

11.2 Required conformance-supported artifacts

ID	Artifact	Required state
GD-001	test run records	generated
GD-002	evidence packets	generated
GD-003	red-line drill records	generated
GD-004	PASS/FAIL/BLOCKED results	recorded
GD-005	unresolved failures	registered
GD-006	claim downgrade if failures exist	applied
GD-007	witness/event refs for material transitions	recorded
GD-008	conformance summary	generated

11.3 Conformance-supported limitation

Conformance-supported does not mean:

safe in all deployments;
legally certified;
provider-compliant in all clouds;
immune to misuse;
security-audited;
production-ready.

12. Required companion documents before Gate B

The following documents are required or should be explicitly deferred before claiming **RG-4-IMPLEMENTATION-READY**.

Priority	File	Gate relevance	Required before Gate B?
P0	CLI_Agent_Raw_Evidence_Sidecar_Profile_v0_1.md	centralizes raw evidence handling	yes
P0	CLI_Agent_Release_Public_Surface_Profile_v0_1.md	controls GitHub/Zenodo/site/PDF/SHA/release branch surface	yes for release-complete; yes before public release automation
P0	CLI_Agent_Registry_Profile_v0_1.md	makes agent admission/permission executable	yes
P0	CLI_Agent_Local_Checker_Profile_v0_1.md	defines local validator/checker handoff	yes
P1	CLI_Agent_Schema_Object_Registry_v0_1.md	canonical object list for extraction	yes
P1	CLI_Agent_Semantic_Validator_Rules_v0_1.md	separates semantic gates from JSON Schema	yes
P1	CLI_Agent_Fixture_Manifest_Profile_v0_1.md	makes fixture pack executable	yes
P1	CLI_Agent_Conformance_Evidence_Packet_Profile_v0_1.md	makes conformance evidence auditable	yes
P1	CLI_Agent_UI_State_Surface_v0_1.md	defines human/c gate visibility	recommended before serious implementation
P2	provider-specific profiles	cloud/provider facts	defer allowed with explicit disclaimer
P2	retention/decay profile	long-term residue control	defer allowed if retention defaults restricted

Priority	File	Gate relevance	Required before Gate B?
P2	cross-c isolation profile	multi-entity isolation	defer allowed unless multi-c implementation is claimed
P2	legal handoff profile	jurisdiction/counsel routing	defer allowed unless legal-sensitive deployment is claimed
P2	cost/budget profile	L4 budgets	defer allowed if basic budget fields remain in CATC

13. Release/public surface requirements

A release-complete package must define the release surfaces affected by publication.

Minimum surfaces:

```

GitHub repository
main/default branch
release tag
release assets
README
INDEX / package index
PDF artifacts
SHA256SUMS
Zenodo record if used
website links if used
restricted bundle if used
public bundle if used

```

13.1 Discoverability rule

Public or archival files intended for ordinary readers **MUST** be discoverable from the default branch or public release entry point.

A document that exists only in a side branch, hidden folder, direct-link-only asset, or internal workspace does not satisfy public discoverability.

13.2 Artifact ordering rule

The release order **SHOULD** be:

```

freeze Markdown
-> final package index
-> final README

```

```
-> final release notes
-> final public/restricted split
-> generate PDFs
-> compute SHA256SUMS
-> final review
-> tag release
-> publish / archive
```

13.3 No premature hash rule

SHA256SUMS MUST NOT be treated as final until all release artifacts are finalized.

Changing PDF, Markdown, manifest, schema, or index after hash generation invalidates release integrity.

14. Public / restricted handling

14.1 Default publication rule

If sensitivity is unclear:

```
default = restricted technical
```

Unknown sensitivity must not default to public.

14.2 Public-capable material

Public-capable material may include:

```
root doctrine
high-level protocol explanation
safe task-contract structure
safe permission/capability distinctions
safe sandbox/worktree discipline
witness concepts without raw evidence
memory gate concepts without private memory
red-line boundaries
safe synthetic fixtures
release notes after hygiene
```

14.3 Restricted material

Restricted material includes or may include:

```
real incident evidence
raw defensive garage residue
```

```
real cloud exposure records
real provider/account/path details
secret boundary records
legal/counsel material
private memory
sealed material
red-line-adjacent internal fixtures
operational canary/signature details
implementation-specific local infrastructure
```

14.4 Restricted document warning

Documents such as defensive emulation, incident response, and secrets/cloud data policy should remain restricted or redacted unless their public version has been reviewed for offensive-conversion risk.

15. Machine-readiness requirements

15.1 JSON Schema is necessary but insufficient

JSON Schema may validate object shape.

It cannot decide:

```
lawful authority;
proportionality;
context safety;
red-line proximity;
provider-specific data handling;
semantic contradiction;
whether a reviewer is genuinely independent;
whether a public release is socially or legally prudent.
```

Therefore implementation readiness requires both:

```
structural schemas
semantic validators
human/`c` gates
witness records
conformance evidence
```

15.2 Minimum machine artifacts before implementation execution

```
schemas/README.md
schemas/SCHEMA_INDEX.json
PACKAGE_MANIFEST.json
fixtures/manifest.json
```

```

expected_results/*.expected.yaml or .json
evidence_packets/templates/*.yaml or .json
semantic_validator_rules/*.yaml or .md
local_checker_profile.md
agent_registry_profile.md
raw_evidence_sidecar_profile.md

```

15.3 Semantic rules that MUST NOT be left to JSON Schema alone

Rule	Why JSON Schema is insufficient
no live external counter-operation	requires context and authorization judgment
no self-approval	requires role/history comparison
no direct memory write	requires target-surface classification
no secret in witness	requires content classification
cloud context not private	requires provider/runtime classification
quorum is not sovereignty	requires authority-chain analysis
public/restricted split	requires disclosure-risk analysis
raw evidence sidecar access	requires privacy/legal/security gate
release discoverability	requires repository/public-surface inspection

16. Readiness record object

Canonical object:

```
CLI_AGENT_READINESS_RECORD
```

16.1 YAML shape

```

cli_agent_readiness_record:
  schema_version: cli-agent-readiness-gate-0.1
  record_id: string
  created_at: string
  package_name: C-Governed CLI Agent Mesh
  package_version: v0.1
  assessed_gate: RG-3-RELEASE-COMPLETE | RG-4-IMPLEMENTATION-READY | RG-5-
IMPLEMENTATION-EXECUTED | RG-6-CONFORMANCE-SUPPORTED
  assessor:
    type: c | human_anchor | reviewer | local_checker | codex | other
    id: string
  source_refs:
    - file: CLI_Agent_RELEASE_NOTES_v0_1.md
      hash_ref: string | null

```

```

- file: CLI_Agent_Contradiction_Register_v0_1.md
  hash_ref: string | null
conditions:
- condition_id: GA-001
  status: pass | fail | partial | blocked | deferred | not_applicable
  evidence_ref: string | null
  note: string
blockers:
- blocker_id: string
  severity: P0 | P1 | P2 | P3
  required_action: string
allowed_claims:
- RGC-DRAFT-COMPLETE
prohibited_claims:
- RGC-IMPLEMENTATION-READY
- RGC-CONFORMANCE-SUPPORTED
decision: pass | fail | partial | blocked
decision_reason: string
witness_required: boolean
witness_ref: string | null

```

16.2 Decision values

Decision	Meaning
pass	Gate conditions satisfied for declared scope.
fail	Gate conditions not satisfied.
partial	Some conditions satisfied; claim must be weaker.
blocked	Critical blocker prevents escalation.

17. Release completion record object

Canonical object:

```
CLI_AGENT_RELEASE_COMPLETION_RECORD
```

17.1 YAML shape

```

cli_agent_release_completion_record:
  schema_version: cli-agent-readiness-gate-0.1
  release_completion_id: string
  package_version: v0.1
  target_release: public | restricted | archival | internal
  markdown_frozen: boolean
  file_inventory_ref: string
  package_index_ref: string
  readme_ref: string
  release_notes_ref: string

```

```

open_issues_ref: string
contradiction_register_ref: string
public_redaction_ref: string
public_bundle_ref: string | null
restricted_bundle_ref: string | null
sensitive_files_reviewed:
  - file: string
    decision: public | redacted_public | restricted | internal
stale_claim_scan:
  status: pass | fail
  notes:
    - string
red_line_pass:
  status: pass | fail
  notes:
    - string
artifact_generation:
  pdfs_generated_after_freeze: boolean
  sha256_generated_after_final_artifacts: boolean
release_decision: pass | fail | blocked
witness_ref: string | null

```

18. Implementation handoff record object

Canonical object:

```
CLI_AGENT_IMPLEMENTATION_HANDOFF_RECORD
```

18.1 YAML shape

```

cli_agent_implementation_handoff_record:
  schema_version: cli-agent-readiness-gate-0.1
  handoff_id: string
  created_at: string
  package_version: v0.1
  executor: codex | local_agent | human | other
  implementation_scope:
    - schemas
    - schema_index
    - package_manifest
    - fixture_manifest
    - expected_results
    - local_checker
    - semantic_validator
    - evidence_packets
  allowed_paths:
    - string
  denied_paths:
    - string
  allowed_commands:
    - string
  denied_commands:
    - string

```



```

required_inputs:
  - file: CLI_Agent_Schema_Object_Registry_v0_1.md
  - file: CLI_Agent_Semantic_Validator_Rules_v0_1.md
  - file: CLI_Agent_Local_Checker_Profile_v0_1.md
output_required:
  - generated_files_list
  - schema_validation_report
  - fixture_manifest_report
  - checker_report
  - unresolved_questions
gates:
  self_approval_allowed: false
  witness_required: true
  human_gate_required_for_release: true
failure_behavior:
  on_missing_registry: hold
  on_red_line_ambiguity: quarantine
  on_protected_path_attempt: freeze
  on_schema_conflict: hold
witness_ref: string | null

```

19. Claim control record object

Canonical object:

```
CLI_AGENT_CLAIM_CONTROL_RECORD
```

19.1 YAML shape

```

cli_agent_claim_control_record:
  schema_version: cli-agent-readiness-gate-0.1
  claim_record_id: string
  package_version: v0.1
  proposed_claim: string
  claim_class: RGC-DRAFT | RGC-DRAFT-COMPLETE | RGC-RELEASE-COMPLETE | RGC-
IMPLEMENTATION-READY | RGC-IMPLEMENTED | RGC-CONFORMANCE-SUPPORTED | RGC-DEPLOYMENT-
READY
  required_gate: RG-0-DRAFT | RG-1-DRAFT-COMPLETE | RG-3-RELEASE-COMPLETE | RG-4-
IMPLEMENTATION-READY | RG-5-IMPLEMENTATION-EXECUTED | RG-6-CONFORMANCE-SUPPORTED
  current_gate: string
  allowed: boolean
  required_downgrade: string | null
  reason: string
  evidence_refs:
    - string
  witness_ref: string | null

```

20. Current package assessment template

This section should be updated during v0.1.1 hygiene patch.

20.1 Current known state before this profile is integrated

```
current_package_assessment:
  package_version: v0.1
  current_state: RG-1-DRAFT-COMPLETE
  release_complete: false
  implementation_ready: false
  implementation_executed: false
  conformance_supported: false
  known_strengths:
    - coherent root doctrine
    - no hard contradiction registered at draft level
    - mature control surfaces for task contract, permission, sandbox, witness, memory
gate, rollback, review, incident, cloud-data, conformance
  known_blockers:
    - package-control registers must be synchronized after newly created files
    - raw evidence sidecar must be centralized
    - release/public surface profile must be added
    - registry profile must be added
    - local checker profile must be added
    - schema object registry must be added
    - semantic validator rules must be added
    - fixture manifest and evidence packet profiles must be added
    - extracted schemas and test runs do not yet exist
  allowed_claims:
    - draft-complete architecture
    - release-completion in progress
    - implementation-readiness preparation
  prohibited_claims:
    - release-complete
    - implementation-ready
    - conformance-passed
    - deployment-ready
```

20.2 Required update rule

After each new companion document is created, this assessment **MUST** be updated or referenced by the Open Issues register and Release Notes.

A stale readiness assessment is itself a release hygiene fault.

21. Stop conditions

The readiness process **MUST** stop or downgrade claims if any of the following occur:

Stop ID	Condition	Required response
STOP-001	hard contradiction discovered	hold package, update contradiction register
STOP-002	red-line wording permits prohibited behavior	freeze release claim, patch wording
STOP-003	sensitive file enters public bundle without redaction review	remove from public bundle, witness issue
STOP-004	raw evidence included in witness/public fixture	quarantine artifact, patch profile
STOP-005	schema object conflict found	hold implementation-ready claim
STOP-006	Codex must infer architecture not specified	return to implementation-readiness preparation
STOP-007	conformance claimed without test results	downgrade claim immediately
STOP-008	release artifact changes after SHA generation	regenerate hashes after final artifact freeze
STOP-009	main/default branch discoverability broken	block public release-complete claim
STOP-010	provider-specific fact asserted without source or profile	remove claim or mark deferred

22. Minimal Codex handoff rule

Codex MUST NOT receive a broad instruction such as:

```
make the package implementation-ready
```

That instruction is too vague.

Codex may receive bounded tasks only after Gate B conditions are defined, such as:

```
extract these exact schemas;
create this exact SCHEMA_INDEX.json;
validate these fixtures against these expected results;
run this local checker;
produce this evidence packet;
update these references only;
do not publish;
do not tag;
do not touch restricted files except listed paths;
stop on contradiction.
```

The executor should be treated as a bounded worker, not as an architect of the readiness gate.

23. Minimal next-step sequence

To move from current draft-complete architecture toward release-complete and implementation-ready status:

1. Create this readiness gate profile.
2. Create Raw Evidence Sidecar Profile.
3. Create Release Public Surface Profile.
4. Create Registry Profile.
5. Create Local Checker Profile.
6. Create Schema Object Registry.
7. Create Semantic Validator Rules.
8. Create Fixture Manifest Profile.
9. Create Conformance Evidence Packet Profile.
10. Create v0.1.1 Hygiene Patch Notes.
11. Update README, Release Notes, Open Issues, Contradiction Register, Package Index, Glossary.
12. Only then write Codex implementation contracts.

24. Readiness checklist summary

24.1 Release-complete checklist

- [] canonical file inventory closed
- [] package index synchronized
- [] README synchronized
- [] release notes synchronized
- [] open issues synchronized
- [] contradiction register synchronized
- [] no stale missing-file claims
- [] public/restricted split applied
- [] sensitive files classified
- [] red-line wording pass complete
- [] raw evidence sidecar created or blocker recorded
- [] release/public surface profile created or blocker recorded
- [] reading order final
- [] Markdown freeze declared
- [] PDFs generated only after freeze
- [] SHA256SUMS generated only after final artifacts
- [] release tag blocked until final review
- [] release decision witnessed or recorded

24.2 Implementation-ready checklist

```
[ ] canonical object registry exists
[ ] schema extraction target list complete
[ ] schema folder layout defined
[ ] stable schema $id convention defined
[ ] SCHEMA_INDEX.json model defined
[ ] package manifest model defined
[ ] semantic validator rules defined
[ ] local checker profile defined
[ ] agent registry profile defined
[ ] fixture manifest model defined
[ ] expected-result model defined
[ ] evidence packet model defined
[ ] conformance runner requirements defined
[ ] red-line behavior machine-checkable
[ ] fail-closed behavior machine-checkable
[ ] witness event minimums defined
[ ] memory gate implementation boundary defined
[ ] provider-specific assumptions profiled or deferred
[ ] raw evidence sidecar handling defined
[ ] Codex handoff constraints written
```

24.3 Conformance-supported checklist

```
[ ] schemas generated
[ ] local checker generated
[ ] fixtures installed
[ ] expected results installed
[ ] test runner executed
[ ] PASS/FAIL/BLOCKED/HELD/QUARANTINED results recorded
[ ] evidence packets generated
[ ] red-line drills recorded
[ ] failures registered
[ ] claims downgraded where needed
[ ] conformance summary generated
```

25. Closing rule

Readiness is not mood.

Readiness is a witnessed transition.

Final rule:

```
If the package cannot show the artifact,
it cannot claim the state.

If the artifact exists but is not linked,
it cannot carry the release.
```

If the test did not run,
it cannot support conformance.

If the boundary is ambiguous,
the claim downgrades.